

Customer Description

The existing **Configurable Edge Pico Platform** provides a modular edge-computing architecture that supports configurable services, sensor integration, communication modules, and deployable AI workloads.

While the platform has demonstrated its capabilities on specific hardware platforms, its architecture remains closely associated with the underlying microcontroller implementation. This limits portability, long-term maintainability, and adoption across different embedded devices.

We want to evolve the platform into a **hardware-independent edge-computing framework built on top of μ T-Kernel**.

The objective is to create a common software platform that allows the same edge application, service, configuration, and AI workload to operate across multiple hardware platforms with minimal modification.

Students should design a layered architecture that separates:

- Application logic
- Edge services
- AI inference services
- Configuration management
- Communication services
- Hardware-specific implementations

The platform should expose consistent APIs and services regardless of the underlying hardware.

Applications should interact with logical capabilities such as:

- Sensors
- Storage
- Communications
- AI inference
- Actuation
- System services

without requiring knowledge of the physical implementation details.

The resulting framework should allow new hardware platforms to be supported by implementing only the necessary hardware adaptation layer while preserving the behavior of existing applications and services.

The completed solution should demonstrate that edge applications can be deployed across different microcontroller platforms running μ T-Kernel without requiring significant application-level modifications.

Motivation

This project builds upon the previous **Configurable Edge Pico Platform**, which focused on configurable edge applications and modular software services.

The next stage of platform evolution is to remove platform dependencies and establish a reusable edge-computing ecosystem that is independent of any particular microcontroller family.

Modern software ecosystems succeed because applications are developed against platform services rather than hardware.

Examples include:

- Android
- Linux
- Zephyr
- ROS
- ESPHome

Developers interact with stable APIs while underlying hardware implementations vary between devices.

This project seeks to apply the same philosophy to Industrial IoT and edge computing by positioning **μ T-Kernel as the common operating foundation** upon which portable edge services can be built.

Rather than developing an application for a specific board, students are expected to design a platform that enables future applications to run across many boards.

Short Product Vision Statement

Develop a **hardware-agnostic edge-computing platform built on μ T-Kernel** that enables portable edge applications, configurable services, and AI workloads to operate across multiple embedded hardware platforms.

Requirements Students Can Infer

Area	Questions students should resolve
Platform Architecture	How should the platform be layered to maximize portability?
Hardware Abstraction	Which hardware capabilities should be abstracted?
Device Framework	How should sensors, actuators, and peripherals be represented?
Service Model	How can services be exposed consistently across hardware platforms?
Configuration System	How can applications be configured independently of hardware?
AI Framework	How can AI inference be exposed through a hardware-independent interface?
Communication Layer	How can networking services remain platform agnostic?
Driver Adaptation	What is required to support a new hardware platform?
Resource Management	How should memory, tasks, and services be managed across platforms?
Deployment Model	How are applications packaged and deployed?
Verification	How can portability be objectively demonstrated?
Extensibility	How easily can future boards be supported?